

SR05 Systèmes et algorithmes répartis

Gestion dynamique de la composition d'un groupe dans une application répartie

Rapport d'étude

Mohamed Amine Jilani

Table des matières

Table des matières **1**

1. Introduction **1**

2. État de l'art — Group Membership 2

2.1 Détection de défaillance : SWIM et la diffusion épidémique 2

2.2 Exploration de réseau et élection : vagues, ondes et terminaison 3

2.3 Autres approches et perspectives 3

2.4 Positionnement de notre étude 3

3. Rappel du projet initial 3

4. Analyse des besoins 4

5. Conception et choix d'architecture 5

5.1 Séparation des responsabilités : le contrôleur NET 5

5.2 Topologie logique en anneau 6

5.3 Couche de transport : le routeur 6

6. Mécanismes mis en place 6

6.1 Admission d'un nouveau participant 7

6.2 Résolution des admissions concurrentes par élection 7

6.3 Diffusion d'une vue commune 7

6.4 Départ volontaire et reconfiguration de l'anneau 8

6.5 Détection des pannes et tolérance aux crashs 8

7. Validation et démonstrations 8

8. Limites et perspectives 9

8.1 Limites 9

8.2 Perspectives 10

9. Répartition du travail et usage de l'intelligence artificielle 10

9.1 Répartition du travail 10

9.2 Usage de l'intelligence artificielle 10

10. Conclusion **11**

Bibliographie 12

1. Introduction

Ce rapport présente une étude menée à la suite d'un projet de systèmes répartis. Ce projet consistait à développer une application répartie de gestion des réservations, dans laquelle plusieurs sites coopèrent pour maintenir des réplicats cohérents d'un même planning. Chaque site dispose d'une copie locale et échange des messages avec les autres participants.

Le projet reposait toutefois sur une hypothèse forte : le nombre de sites était fixé au démarrage et la liste des membres ne pouvait pas évoluer. Tout ajout ou retrait imposait une reconfiguration manuelle du réseau et un redémarrage de l'application.

Cette limite constitue le point de départ de l'étude. La question posée est la suivante : comment permettre à la liste des participants d'évoluer sans perturber le fonctionnement de l'application existante ? Ce problème relève de la gestion de la composition d'un groupe (*group membership*). L'objectif est d'ajouter un mécanisme capable de gérer l'arrivée d'un nouveau site, le départ volontaire d'un membre et la disparition d'un participant en panne, tout en préservant les algorithmes répartis déjà présents.

Pour cela, nous avons conçu un contrôleur réseau nommé NET. Il prend en charge les admissions, les départs, la diffusion des changements de vue et la reconfiguration du réseau.

Le rapport dresse d'abord l'état de l'art, puis rappelle l'architecture du projet initial et analyse les besoins de l'étude. Il présente ensuite les choix de conception, avant de détailler les mécanismes mis en place, leur validation, et enfin les limites et perspectives du travail.

2. État de l'art — Group Membership

La gestion de la composition d'un groupe est un problème central des systèmes répartis. Chaque participant doit connaître l'ensemble des membres actifs, appelé une vue (*view*), et tous les sites doivent partager la même. Maintenir cette vue est difficile, car des participants tombent en panne, partent ou rejoignent le réseau en cours d'exécution. Cette section présente les travaux qui ont guidé notre étude.

2.1 Détection de défaillance : SWIM et la diffusion épidémique

Détecter qu'un participant est tombé en panne est une brique de base de la gestion de groupe. Les premières solutions échangeaient des battements de cœur (*heartbeats*) entre tous les sites, mais cette approche ne passe pas à l'échelle. Le protocole SWIM (*Scalable Weakly-consistent Infection-style Process Group Membership Protocol*) répond à ce problème [1]. Il sépare deux fonctions souvent confondues, la détection de défaillance et la diffusion des changements de membre.

La détection repose sur un sondage aléatoire périodique. Chaque nœud interroge un autre nœud choisi au hasard. En cas de silence, il demande à d'autres membres de sonder le suspect à sa place. Ce sondage indirect (indirect probing) limite les fausses détections. Le membre suspecté n'est pas déclaré mort tout de suite, et un numéro d'incarnation (*incarnation number*) lui permet de prouver qu'il est toujours actif. La charge réseau par membre reste constante quelle que soit la taille du groupe.

La diffusion des changements suit un modèle épidémique, aussi appelé gossip. Les informations s'accrochent aux messages de sondage déjà échangés et se propagent de proche en proche. Ce principe explique l'adoption de SWIM dans des systèmes en production comme Cassandra ou Consul [5]. Une implémentation dans la base de données Tarantool en illustre les choix concrets [4].

2.2 Exploration de réseau et élection : vagues, ondes et terminaison

Pour coordonner une opération répartie, il faut souvent désigner un site initiateur, puis savoir quand l'opération est terminée. Les algorithmes d'exploration de réseau répondent à ce besoin [6]. Un initiateur réveille les autres sites par une cascade de messages. Lorsque l'information remonte ensuite vers lui, il sait que tous les sites ont été atteints. Ce retour vers l'initiateur sert donc à détecter la terminaison.

L'élection s'appuie sur ces mécanismes. Elle consiste à désigner un site unique parmi des sites identiques au départ. Pour cela, les algorithmes utilisent les identités des sites, qui forment un ordre total. Le site élu est en général celui qui possède la plus petite identité.

Sur un anneau, l'élection se fait simplement par un calcul de minimum [6]. Cette solution est connue sous le nom d'algorithme de Chang et Roberts. Chaque candidat envoie son identité sur l'anneau. Un site ne retransmet que les identités plus petites que la sienne. Lorsqu'un candidat voit sa propre identité revenir après un tour complet, il sait qu'aucune identité plus petite n'existe. Il est alors élu. Le tour de l'anneau joue ainsi le rôle de sélection et de détection de terminaison.

Cette technique correspond directement à notre mécanisme d'admission. Lorsqu'un site veut modifier la composition d'un groupe, il lance une élection qui circule dans l'anneau. Le retour du message à l'initiateur lui indique que tous les participants ont été consultés et que la modification peut être appliquée.

2.3 Autres approches et perspectives

D'autres travaux explorent des directions complémentaires à la nôtre. Sur le plan théorique, des services de groupe ont été proposés pour les réseaux dynamiques [2]. Ils reviennent seuls à un état cohérent après une perturbation et garantissent un diamètre de groupe borné, ce qui les rend plus robustes que notre approche. Sur le plan sécurité, des travaux récents protègent la composition du groupe par des moyens cryptographiques [3]. Ils combinent un accord de clé de groupe contributif, un schéma résistant aux attaques quantiques et une chaîne de blocs pour enregistrer les arrivées et les départs. Ces deux directions sont orthogonales à notre étude, mais elles en montrent les prolongements possibles.

2.4 Positionnement de notre étude

Notre étude combine plusieurs de ces idées. La détection de défaillance s'inspire de SWIM, dont nous reprenons la surveillance périodique. La coordination des changements de groupe repose quant à elle sur une élection sur anneau par calcul du minimum, qui sérialise les modifications de la composition. Une vue numérotée garantit ensuite que tous les sites partagent le même groupe.

Notre choix de cohérence est plus fort que celui de SWIM qui accepte une vue faiblement cohérente et une convergence progressive. Nous avons au contraire retenu une vue fortement cohérente, où une seule modification est appliquée à la fois. Ce choix se justifie par notre échelle, car le système ne comporte que quelques sites. Il privilégie la simplicité et la cohérence immédiate, au prix du passage à l'échelle visé par SWIM. Les écarts précis entre notre détection de panne et celle de SWIM sont détaillés en section 6.5.

3. Rappel du projet initial

L'étude s'appuie sur le projet réalisé en début de semestre. Ce rappel en présente l'architecture et les mécanismes principaux. Il décrit le socle technique sur lequel repose la gestion dynamique des participants.

Le projet consistait à développer une application répartie de gestion de réservations. Chaque participant possédait une copie locale du planning et pouvait demander une réservation. L'objectif principal était de garantir la cohérence des réplicats malgré les accès concurrents des utilisateurs. La ressource critique était définie par le couple salle et date. Ce choix autorise des réservations parallèles sur des salles différentes.

L'architecture séparait la logique applicative des mécanismes répartis. Chaque site reposait sur deux entités. APP représentait l'application utilisateur et la gestion des réservations. CTL assurait les mécanismes de contrôle et les échanges liés aux algorithmes répartis. Une interface web permettait de créer une réservation, de consulter l'état du système et de déclencher un instantané réparti (*snapshot*). L'organisation générale d'un site pouvait se résumer ainsi :

$$APP \leftrightarrow CTL \leftrightarrow Réseau \leftrightarrow CTL \leftrightarrow APP$$

Le projet mettait en œuvre plusieurs concepts classiques des systèmes répartis. La cohérence du planning reposait sur l'exclusion mutuelle répartie inspirée de l'algorithme de Ricart-Agrawala. Les événements étaient ordonnés à l'aide d'horloges logiques de Lamport et d'horloges vectorielles. L'instantané réparti inspiré de l'algorithme de Chandy-Lamport permettait de capturer l'état local des participants grâce à des messages marqueurs (*markers*).

Les communications entre sites étaient simulées par des tubes FIFO reliés par des scripts shell. Les participants formaient un réseau complet. Chaque site communiquait donc directement avec tous les autres membres du groupe.

Cette architecture validait les mécanismes répartis étudiés, mais reposait sur l'hypothèse d'une liste de participants fixe, déjà évoquée en introduction. C'est cette limite que l'étude lève.

4. Analyse des besoins

Comme indiqué en introduction, le projet initial supposait une liste de participants fixe, ce qui simplifiait le développement mais limitait fortement l'évolutivité de l'application. Dans un système distribué réel, la composition du groupe varie au cours du temps. Un nouveau site peut vouloir rejoindre le réseau. Un membre peut décider de le quitter. Un participant peut aussi devenir inaccessible à la suite d'une panne. Le système doit alors s'adapter à ces changements sans interrompre son fonctionnement. Cette adaptation soulève plusieurs difficultés.

La première difficulté concerne la cohérence de la liste des membres. Tous les participants doivent partager la même vision du groupe. Sans cela, leurs comportements divergent. Supposons par exemple que certains sites considèrent un participant comme toujours présent, alors que d'autres l'ont déjà retiré. Les échanges de messages risquent d'échouer ou de produire des états incohérents.

La deuxième difficulté apparaît lorsque plusieurs modifications de groupe surviennent en même temps. Deux participants peuvent tenter de rejoindre le réseau

simultanément. Une arrivée peut aussi se produire pendant le départ d'un autre membre. En l'absence de coordination, ces opérations concurrentes conduisent à des vues incompatibles.

Une autre difficulté est liée aux mécanismes déjà présents dans le projet. Les horloges vectorielles, la diffusion des messages et les instantanés répartis reposent tous sur une liste cohérente des membres. La taille d'une horloge vectorielle dépend même directement du nombre de sites. Toute modification de la composition du groupe doit donc être propagée de façon contrôlée.

La dernière difficulté concerne la nature du départ. Le système doit distinguer deux cas. Dans le premier, le participant quitte volontairement le réseau et peut prévenir les autres membres avant de partir. Dans le second, il disparaît brutalement à cause d'une panne : le système doit alors détecter cette absence de lui-même, puis reconfigurer le réseau sans intervention extérieure.

L'étude vise donc à concevoir un mécanisme de gestion dynamique des participants. Celui-ci doit assurer :

- l'admission de nouveaux membres ;
- le retrait volontaire d'un participant ;
- la détection et la gestion des pannes ;
- le maintien d'une vue cohérente du groupe ;
- la préservation des mécanismes répartis déjà présents dans le projet.

Pour répondre à ces besoins, nous avons choisi d'introduire un composant dédié à la gestion du réseau et de la composition du groupe. Les choix d'architecture correspondant sont présentés dans la section suivante.

5. Conception et choix d'architecture

Cette section présente les choix retenus pour rendre la composition du groupe dynamique. Trois décisions structurent l'étude. La première sépare la gestion du réseau de la logique applicative. La deuxième adopte une topologie en anneau. La troisième introduit un routeur pour la simulation des communications. Chaque choix est présenté avec l'alternative écartée, puis avec les raisons qui l'ont motivé.

5.1 Séparation des responsabilités : le contrôleur NET

L'objectif principal est d'ajouter la gestion dynamique des participants sans modifier en profondeur l'application existante. Nous avons donc choisi de séparer la gestion du réseau de la logique applicative.

Dans le projet initial, les contrôleurs CTL communiquaient directement entre eux. Cette approche suffisait tant que le nombre de participants restait fixe. L'ajout ou le retrait d'un membre aurait toutefois imposé d'enrichir fortement le contrôleur CTL. Or ce contrôleur était dédié aux mécanismes répartis liés aux réservations et aux instantanés. Y intégrer la gestion du groupe aurait nui à sa lisibilité.

Pour conserver une architecture modulaire, nous avons introduit un nouveau composant appelé NET. Ce contrôleur prend en charge la gestion du réseau et de la composition du groupe. L'admission d'un participant, son départ, la diffusion des changements de vue et la détection des pannes sont ainsi isolés du reste de l'application.

L'architecture initiale était la suivante :

APP ↔ CTL ↔ Réseau ↔ CTL ↔ APP

Elle devient :

APP ↔ CTL ↔ NET ↔ Réseau ↔ NET ↔ CTL ↔ APP

Dans cette organisation, l'application APP continue à communiquer uniquement avec son contrôleur local CTL. Les mécanismes du projet précédent restent concentrés dans CTL. Toutes les opérations liées au réseau passent désormais par NET. Cette séparation améliore la lisibilité de l'architecture. Elle facilite la maintenance. Elle permet surtout de faire évoluer la gestion des participants sans toucher à la logique applicative.

5.2 Topologie logique en anneau

Le projet initial utilisait un réseau complet construit à l'aide de tubes nommés. Chaque participant possédait une liaison directe avec tous les autres membres.

Cette solution est simple à mettre en œuvre pour un faible nombre de participants, mais devient vite difficile à maintenir lorsque le groupe évolue. L'ajout d'un participant impose de créer une connexion avec chacun des membres déjà présents. La suppression d'un participant oblige à reconfigurer plusieurs liaisons.

Pour simplifier la gestion dynamique du réseau, nous avons retenu une topologie logique en anneau. Chaque participant ne conserve alors que deux voisins directs : un prédécesseur et un successeur. Cette organisation réduit fortement le nombre de connexions à maintenir. L'arrivée d'un participant ne modifie que quelques liaisons locales, et le départ d'un membre ne nécessite que la reconnexion de ses voisins directs. L'anneau se prête en outre bien aux mécanismes de diffusion et d'élection présentés plus loin : un message peut circuler le long de l'anneau tout en atteignant l'ensemble des participants.

5.3 Couche de transport : le routeur

Au début de l'étude, la communication entre les participants reposait uniquement sur des scripts shell reliant directement les tubes nommés des différents sites. Cette approche fonctionnait pour une topologie fixe, mais chaque arrivée ou départ exigeait d'arrêter puis de recréer plusieurs processus de redirection. Cette manipulation rendait la gestion dynamique du groupe difficile et compliquait les expérimentations.

Pour simplifier les communications, nous avons introduit un composant supplémentaire appelé routeur. Son rôle est de centraliser les échanges et d'acheminer chaque message vers sa destination. Chaque instance NET s'enregistre auprès du routeur au démarrage à l'aide d'un message REGISTER. Le routeur construit alors une table dynamique qui associe chaque identifiant de site à son point d'entrée. Un message est ensuite acheminé soit vers un destinataire précis grâce à un champ to, soit vers tous les sites avec la valeur to=ALL.

Ce choix présente plusieurs avantages. Les connexions n'ont plus à être modifiées lors d'un changement de topologie. L'ajout d'un participant ne demande plus de reconfigurer l'ensemble des liaisons. La logique réseau se concentre dans un composant unique. Les mécanismes d'admission, de départ et de diffusion deviennent plus simples à implémenter.

Un point mérite d'être souligné. Le routeur ne participe pas aux algorithmes répartis. Il agit uniquement comme une couche de transport destinée à faciliter la simulation. Les décisions d'admission, d'élection et de reconfiguration restent prises par les participants eux-mêmes. Le système conserve donc son caractère distribué.

6. Mécanismes mis en place

Cette section décrit les mécanismes ajoutés pour gérer la composition dynamique du groupe: arrivée d'un participant, résolution des demandes concurrentes, synchronisation de l'état, diffusion d'une vue commune, départ volontaire et détection des pannes.

6.1 Admission d'un nouveau participant

Pour rejoindre le réseau, un site candidat contacte un membre déjà présent, qui devient le point d'entrée de l'admission. Il lui envoie un message JOIN_REQUEST contenant son identité. Le membre contacté ne l'accepte pas immédiatement : il lance d'abord une élection dans l'anneau pour vérifier qu'aucune autre admission plus prioritaire n'est en cours (voir section 6.2).

Une fois l'admission validée, le candidat reçoit un JOIN_ACCEPT contenant son prédécesseur et son successeur, et s'insère dans l'anneau. Nous le plaçons juste après le membre qui a accepté sa demande : si A accepte D, alors D est inséré entre A et l'ancien successeur de A, et l'anneau passe par exemple de A→B→C→A à A→D→B→C→A. Ce choix limite les modifications, car le site d'accueil connaît déjà son successeur. La topologie est ensuite mise à jour par un message RECONFIGURE_ADD, où seuls le site d'accueil et son ancien successeur modifient leurs références.

L'admission seule ne suffit pas : le nouveau participant doit récupérer l'état courant pour être opérationnel. Le coordinateur lui envoie donc un message SYNC contenant le planning avec ses réservations, la topologie de l'anneau et la liste des membres. Le nouveau site initialise alors ses structures locales et possède le même état que les autres. Sans cette étape, il ignorerait l'historique des réservations et risquerait de réserver un créneau déjà occupé. La nouvelle composition est enfin diffusée par MEMBER_LIST.

6.2 Résolution des admissions concurrentes par élection

Plusieurs participants peuvent tenter de rejoindre le réseau au même moment. Sans coordination, deux modifications simultanées de la composition produiraient des vues incompatibles. Nous n'autorisons donc qu'une seule modification à la fois, au moyen d'une élection inspirée du principe d'élection par extinction vu en cours. Chaque admission porte un identifiant unique de la forme ADM-hôte-nouveauSite ; par exemple, si D entre via A, l'identifiant est ADM-A-D.

Lorsque deux élections circulent en même temps, une règle déterministe les départage : l'identifiant le plus petit l'emporte. Si D contacte A et E contacte B au même instant, A diffuse ADM-A-D et B diffuse ADM-B-E. Comme ADM-A-D est le plus petit, A devient seul coordinateur ; l'admission de E est annulée puis relancée plus tard. Lorsque le message d'élection revient à son initiateur sans avoir été éliminé, celui-ci sait que tout l'anneau a été consulté, procède à l'admission, puis libère le groupe. Les admissions sont ainsi traitées de façon séquentielle, ce qui garantit une vue cohérente à tout instant.

6.3 Diffusion d'une vue commune

Toute admission ou tout retrait modifie la composition du groupe, que tous les sites doivent partager. Nous introduisons pour cela la notion de vue : l'ensemble des membres à un instant donné, identifié par un numéro de version incrémenté à chaque modification. Le site responsable diffuse la nouvelle composition et son numéro de version à l'ensemble du réseau, et chaque participant met à jour sa liste locale. Cette mise à jour conditionne le bon

fonctionnement des mécanismes répartis du projet, comme rappelé dans l'analyse des besoins. Le numéro de version permet en outre d'ignorer une information périmée : un participant ne revient jamais à une vue plus ancienne.

6.4 Départ volontaire et reconfiguration de l'anneau

Un membre peut quitter le réseau de son plein gré. Comme il est encore actif et connaît ses voisins, la reconfiguration est simple. Il diffuse un message LEAVE_REQUEST contenant son identité, son prédécesseur et son successeur. Les voisins directs adaptent leurs références : le prédécesseur adopte le successeur du site parti, et inversement. L'anneau reste connecté ; par exemple, si B quitte A→B→C→A, alors A est directement relié à C, sans affecter le reste du réseau. Une nouvelle vue est ensuite diffusée par MEMBER_LIST, puis le site sortant devient passif. Comme les autres ont déjà reçu la vue sans lui, personne n'attend de réponse de sa part et aucune fausse suspicion de panne n'est déclenchée.

6.5 Détection des pannes et tolérance aux crashs

Un site peut aussi disparaître brutalement à la suite d'une panne, sans pouvoir prévenir personne. Le système doit alors détecter cette absence par lui-même, grâce à un mécanisme de heartbeat. Chaque site actif envoie un message PING aux membres connus toutes les cinq secondes ; le destinataire répond par un PONG, qui met à jour la date du dernier contact dans une structure LastSeen.

Si un membre ne répond plus depuis plus de quinze secondes, il est suspecté défaillant. Cette suspicion repose sur un unique seuil de délai, sans état SUSPECT formel ni phase intermédiaire. Le site détecteur diffuse alors un MEMBER_REMOVED contenant l'identité du site disparu et les références de ses voisins. À la réception, tous les participants le retirent de leur vue et reconnectent son prédécesseur à son successeur, sans intervention manuelle. Comme le site disparu ne peut plus fournir sa topologie, chaque participant maintient une table associant à chaque membre ses voisins, ce qui permet de réparer l'anneau en son absence.

Comme annoncé en section 2, ce mécanisme s'inspire de SWIM, dont il reprend la surveillance périodique, dans une version simplifiée : la surveillance porte sur tous les membres connus par PING/PONG directs, et une panne est déclarée à l'expiration d'un délai. Contrairement à SWIM, nous n'utilisons ni sondage aléatoire, ni vérification indirecte de type ping-req, ni état SUSPECT. Ce choix privilégie la simplicité et la lisibilité pédagogique, tout en gardant le système fonctionnel lorsqu'un participant disparaît sans prévenir.

7. Validation et démonstrations

Les mécanismes ont été validés par des scénarios de démonstration. Un script (ctl-net.sh) lance le routeur, puis plusieurs sites reliés en anneau, chacun exécutant ses contrôleurs NET et CTL et exposant une interface web locale. Les échanges sont observés à la fois dans l'interface et dans les journaux des contrôleurs.

Le réseau démarre avec trois sites A, B et C. L'état réseau confirme la composition initiale et l'anneau qui les relie. Une réservation est créée depuis A (par exemple la salle A le 22 avril 2026 de 10 h à 11 h) et se retrouve bien répliquée sur B et C. Ce point fixe l'état du planning avant toute modification de la composition du groupe et sert de référence pour la suite.

Admission d'un participant. Un site D rejoint le groupe via A : une élection est lancée, D est accepté, et l'anneau passe de A→B→C→A à A→D→B→C→A. La nouvelle vue contient A, B, C et D, ce qui valide la chaîne complète d'admission jusqu'à la diffusion de la composition. Le planning de D contient déjà la réservation créée avant son arrivée, preuve que le message SYNC a transmis l'état courant. Pour le confirmer, D tente de réserver le même créneau : la réservation est refusée, ce qui montre que D partage réellement l'état du groupe et participe au contrôle des conflits.

Admissions concurrentes. D demande son admission via A et E via B au même moment. Les deux élections se propagent, une seule l'emporte selon la règle de l'identifiant le plus petit, et la seconde admission est écartée puis relancée. Ce scénario valide la sérialisation des modifications et le maintien d'une vue unique.

Départ volontaire. Un membre quitte le réseau et diffuse un message de retrait contenant ses voisins, qui se reconnectent. L'état réseau confirme sa disparition de l'anneau et son passage à l'état inactif, ce qui valide la reconfiguration locale de la topologie.

Disparition d'un site. Un participant est arrêté brutalement (pkill -9 ou ctl+C depuis le terminal de site ajouté). Ses voisins ne reçoivent plus de PONG ; passé le délai, il est déclaré défaillant, une suppression est diffusée et l'anneau se répare automatiquement. Ce scénario valide la détection des pannes par heartbeat et la tolérance aux crashs.

Cohérence des snapshots après évolution du groupe. Après l'ajout de D, un snapshot réparti est déclenché. Les états locaux sont sauvegardés en fichiers JSON, tous contenant la réservation de référence avec A comme site créateur. Le snapshot reste donc cohérent malgré l'évolution du groupe, et le nouveau participant y est correctement intégré.

Ces scénarios montrent que le système gère une composition de groupe variable, sans arrêt de l'application ni reconfiguration manuelle, tout en préservant la cohérence du planning et l'exploitabilité des snapshots. Dans cette version, certains événements distribués restent toutefois surtout visibles dans les journaux des contrôleurs, leur remontée complète vers l'interface web demeurant partielle.

8. Limites et perspectives

8.1 Limites

Le travail présenté comporte plusieurs limites. La première tient à la simulation du réseau : les communications passent par un routeur central, qui facilite les expérimentations mais constitue un point unique de défaillance. Le système ne repose pas encore sur de vraies communications réseau. La détection des pannes est elle aussi simplifiée, selon les écarts avec SWIM décrits en section 6.5. Ces choix privilégient la simplicité et la lisibilité, au prix d'une sensibilité accrue aux faux positifs et d'une moindre capacité de passage à l'échelle. Une autre limite concerne les horloges vectorielles, dont la taille n'est pas encore ajustée automatiquement lors d'un changement de vue. Cet ajustement reste à traiter. L'élection suppose par ailleurs que l'anneau reste stable pendant son déroulement. Une panne survenant au cours d'une élection n'est pas explicitement gérée dans cette version. Enfin, la remontée des événements distribués vers l'interface web demeure partielle : les messages d'admission, de départ ou de panne sont surtout visibles dans les journaux des contrôleurs.

8.2 Perspectives

Plusieurs pistes prolongeraient naturellement ce travail. La plus directe consiste à redimensionner automatiquement les horloges vectorielles à chaque changement de vue, afin qu'elles suivent l'évolution réelle du groupe.

La détection des pannes pourrait ensuite se rapprocher du protocole SWIM complet. Cela passerait par l'introduction d'un état intermédiaire SUSPECT avant la déclaration de mort, d'un numéro d'incarnation pour réfuter une suspicion erronée, d'un sondage aléatoire et d'une vérification indirecte de type ping-req. Ces ajouts réduiraient les faux positifs et amélioreraient le passage à l'échelle.

Le routeur central pourrait également être remplacé par de véritables communications réseau, ce qui supprimerait le point unique actuel et rapprocherait le prototype d'un système réparti réel.

Une dernière piste consiste à faire remonter l'ensemble des événements distribués jusqu'à l'interface web, afin d'offrir une visualisation en temps réel de la composition du groupe, des admissions, des départs et des pannes.

9. Répartition du travail et usage de l'intelligence artificielle

Conformément aux consignes, nous précisons l'usage fait des outils d'intelligence artificielle. Ces outils ont servi d'assistance ponctuelle, et non de substitut au travail réalisé.
Pendant le développement, l'IA a aidé à :

- Produire des exemples de scripts shell pour automatiser le lancement des sites et les commandes shells ;
- Développer certaines parties de l'interface web en Javascript, HTML, CSS ;
- Avoir des informations précises concernant la création d'un fichier json.

Concernant la partie état de l'art, les recherches et la rédaction n'ont pas fait appel à l'usage d'intelligence artificielle. Les articles ont été recherchés, lus et synthétisés manuellement. Cependant certaines parties de la rédaction ont été aidées par intelligence artificielle afin d'améliorer certaines tournures de phrase.

10. Conclusion

Cette étude prolonge un projet de systèmes répartis dont la liste des participants était figée et répond à une question précise : comment faire évoluer cette liste pendant l'exécution sans perturber l'application existante.

Pour y parvenir, nous avons introduit un contrôleur réseau dédié, NET, séparé de la logique applicative. Nous avons remplacé le réseau complet par une topologie logique en anneau, plus simple à reconfigurer. Un routeur assure la simulation des communications sans intervenir dans les algorithmes répartis.

Sur cette base, le système gère désormais l'arrivée d'un participant, son départ volontaire et sa disparition à la suite d'une panne. Les admissions concurrentes sont résolues par une élection sur anneau, une vue numérotée garantit que tous les sites partagent la même composition du groupe, et la synchronisation du planning permet à un nouveau site de devenir immédiatement opérationnel. La détection des pannes s'inspire de SWIM, dans une version adaptée à notre échelle.

Les principales perspectives, détaillées en section 8, portent sur le redimensionnement des horloges vectorielles, le remplacement du routeur par de vraies communications réseau et un rapprochement avec le protocole SWIM complet. Plus largement, cette étude nous a permis de mesurer les difficultés concrètes de la gestion dynamique d'un groupe et illustre le compromis permanent des systèmes répartis entre cohérence, simplicité et passage à l'échelle.

Bibliographie

[1] A. Das, I. Gupta, A. Motivala. « SWIM: Scalable Weakly-consistent Infection-style Process Group Membership Protocol ». Proceedings of the International Conference on Dependable Systems and Networks (DSN), 2002.

[2] B. Ducourthial, S. Khalfallah, F. Petit. « Best-effort Group Service in Dynamic Networks ». 2008. arXiv:0810.3836.

[3] R. P. S, B. Rudra. « Advancing Security and Scalability: A Protocol Extension for Dynamic Group Membership Management ». Journal of Machine and Computing, vol. 5, n° 3, 2025. DOI : 10.53759/7669/jmc202505151.

[4] V. Shpilevoy. « SWIM: Protocol to Build a Cluster ». FOSDEM, 2020.

[5] « Membership Protocols: SWIM, Gossip, and How Distributed Systems Track Who's Alive ». Codelit.io.

[6] Polycopié du cours SR05, Université de Technologie de Compiègne, version 2019, chapitres 8 (Exploration de réseaux) et 9 (Élection).